

С ДЛЯ ПИТОНISTОВ • ДЕНЬ 3

Куча

Почему `append` — амортизированная $O(1)$?

Вопрос дня

```
>>> import sys
>>> a = []
>>> sys.getsizeof(a)
56
>>> for i in range(9):
...     a.append(i)
...     print(len(a), sys.getsizeof(a))
1 88
2 88
3 88
4 88
5 120
```

Размер меняется скачками.
Четыре элемента помещаются
в 88 байт, пятый требует 120.

Вопрос дня

```
>>> import sys
>>> a = []
>>> sys.getsizeof(a)
56
>>> for i in range(9):
...     a.append(i)
...     print(len(a), sys.getsizeof(a))
1 88
2 88
3 88
4 88
5 120
```

Размер меняется скачками.
Четыре элемента помещаются
в 88 байт, пятый требует 120.

Между скачками `append` только
записывает число в готовое
место. На скачке он выделяет
новый блок и переносит всё,
что уже накоплено.

Сегодня разбирается, откуда
берётся этот блок, кто его
переносит и почему такой
`append` всё равно считают
операцией за $O(1)$.

РАЗДЕЛ 1

Куча

Чего не умеет стек

Вчерашняя память вся была на стеке, и у неё три ограничения.

1. **Размер известен при компиляции.** `int a[n]` с переменной `n` на курсе запрещён флагом `-Wvla`.
2. **Размер невелик.** `ulimit -s` даёт 8192 килобайта на все кадры сразу; локальный массив на четыре миллиона `int` — это 16 МБ и `Segmentation fault`.
3. **Жизнь равна вызову.** После `return` кадр занимает следующая функция, а отданный наружу адрес становится висячим.

Главное

Куча снимает все три ограничения сразу. Размер запрашивается во время работы, он ограничен памятью машины, а блок живёт до тех пор, пока его не освободят явно.

malloc и free

```
int *a = malloc(n * sizeof *a);    // просим n * 4 байта
if (a == NULL) { /* памяти нет */ }
...
free(a);                          // возвращаем тот же адрес
```

`malloc` принимает число **байт** и возвращает адрес начала блока либо `NULL`. Проверка обязательна: работа по нулевому указателю — UB, и вчерашнее демо показывало, как это выглядит.

Пишите `sizeof *a`, а не `sizeof(int)`

Запись `sizeof *a` читается «размер того, на что показывает `a`». Она переживает смену типа: если `int` поменять на `double`, строка останется верной. Разыменования здесь не происходит — `sizeof` не вычисляет своё выражение, а только смотрит на его тип.

Где это лежит

```
&on_stack = 0x7ffc41118594  
heap      = 0x56375d889010  
&heap     = 0x7ffc411185a0
```

Указатель `heap` лежит на стеке. Блок, на который он показывает, — нет.

растёт вниз



0x7ff...

растёт вверх



0x563...

Куча — отдельная область, и адреса в ней на нашей машине заметно меньше стековых. Крупные блоки библиотека берёт у операционной системы отдельно, и такой блок может оказаться в стороне от остальных: полагаться на расположение не следует ни в одном из случаев.

Что известно про блок

```
int *a = malloc(4 * sizeof *a);  
printf("%zu\n", sizeof a);    // 8 — размер указателя, и только
```

Длину по-прежнему носите за собой: функции «сколько байт в этом блоке» в стандартной библиотеке нет. Всё, что вчера говорилось про массив и его длину, относится и к куче.

Содержимое свежего блока стандарт не определяет. Читать его до первой записи нельзя, и увиденное там не детерминированно.

```
int *z = calloc(4, sizeof *z);    // то же самое, но обнулено
```

`calloc` принимает количество и размер отдельно и заполняет блок нулями. Он же проверяет произведение на переполнение — у `malloc` этой проверки нет.

free и владение

`free` принимает адрес, который вернул `malloc`, `calloc` или `realloc`. Никакой другой адрес туда передавать нельзя: ни середину блока, ни адрес локальной переменной.

```
free(a);  
a = NULL;      // привычка, которая делает ошибку заметной
```

После `free` указатель хранит прежнее число, но блока по этому адресу больше нет. Обнуление превращает будущую ошибку из тихой в падение на первом же обращении.

Правило владения

У каждого блока ровно один владелец — та часть программы, которая обязана его освободить. Владелец может смениться, но он всегда один, и `free` вызывается ровно один раз.

Три вида памяти

	живёт	размер задаётся
<code>int a[10];</code> внутри функции	до конца вызова	при компиляции
<code>static int a[10];</code>	до конца программы	при компиляции
<code>malloc</code>	до <code>free</code>	во время работы

Третья строка — единственная, где программа сама решает и сколько взять, и когда отдать.

Как это выглядит в Python

В Python все объекты живут в куче, а решение об освобождении принимает интерпретатор: он считает ссылки и освобождает объект, когда счётчик обнуляется. `free` в Python называется `del` и уменьшает счётчик, а не освобождает память.

Теперь функция может вернуть память

Указатель на стек

```
int *f(void) {  
    int a[8];  
    return a;    // висячий  
}
```

Указатель на кучу

```
int *f(size_t n) {  
    int *a = malloc(n * sizeof *a);  
    return a;    // живёт до free  
}
```

Разница в том, где память. Кадр вызова исчезает при выходе из функции, блок в куче — нет.

Главное

Функция, возвращающая `malloc`, передаёт вместе с адресом обязанность вызвать `free`. Записать это в языке негде, поэтому пишется в комментарии — и на защите спрашивается, кто именно владелец.

РАЗДЕЛ 2

Четыре ошибки работы с динамической памятью

```
int *a = malloc(4 * sizeof *a);  
/* free не вызван */
```

Программа при этом работает совершенно правильно, печатает верный ответ и завершается с нулевым кодом возврата. Без санитайзера утечка ничем себя не выдаёт — в этом её главное свойство.

Что говорит санитайзер

```
ERROR: LeakSanitizer: detected memory leaks  
Direct leak of 40 byte(s) in 1 object(s) allocated from:  
    #1 in sum_of_squares 04-leak.c:19  
SUMMARY: AddressSanitizer: 60 byte(s) leaked in 2 allocation(s).
```

Отчёт снимается при завершении программы и называет строку, в которой блок был выделен, для которого не вызвано `free`.

Обращение после освобождения

```
free(a);  
printf("%d\n", a[0]);
```

```
warning: pointer 'a' used after 'free' [-Wuse-after-free]  
note: call to 'free' here
```

Компилятор предупреждает сам, пока обе строки видны ему рядом. В программе посложнее он молчит, и остаётся санитайзер:

Неопределённое поведение

```
ERROR: AddressSanitizer: heap-use-after-free on address 0x7b1d137e0010  
READ of size 4 at 0x7b1d137e0010 thread T0  
0x7b1d137e0010 is located 0 bytes inside of 16-byte region  
freed by thread T0 here: ...  
previously allocated by thread T0 here: ...
```

Три места сразу: где прочитали, где освободили, где выделяли.

Освобождение дважды

```
free(a);  
free(a);
```

Единственная из четырёх ошибок, которую замечает сама стандартная библиотека, без всякого санитайзера:

```
free(): double free detected in tcache 2  
Aborted (core dumped)
```

Аллокатор хранит служебные данные рядом с блоками и по ним видит, что этот адрес уже возвращён. Санитайзер называет то же самое короче:

```
ERROR: AddressSanitizer: attempting double-free on 0x7b85423e0010
```

Обнуление указателя после `free` закрывает эту ошибку целиком:
`free(NULL)` — законная операция, которая ничего не делает.

Запись за границу блока

```
int *a = malloc(4 * sizeof *a);  
a[4] = -1;      // блок рассчитан на четыре элемента
```

Без санитайзера запись проходит, и прочитать оттуда можно то же, что записали. Испорчены при этом служебные данные аллокатора, лежащие сразу за блоком, а проявится это на каком-нибудь следующем `malloc`.

Неопределённое поведение

```
ERROR: AddressSanitizer: heap-buffer-overflow on address 0x7b27ebfe0020  
WRITE of size 4 at 0x7b27ebfe0020 thread T0  
0x7b27ebfe0020 is located 0 bytes after 16-byte region
```

Отличие от вчерашнего `stack-buffer-overflow` — одно слово в названии.

Как это выглядит без санитайзера

ошибка	компилятор	запуск без санитайзера
утечка	молчит	верный ответ, код возврата 0
использование после <code>free</code>	предупреждает	печатает мусор, код возврата 0
двойной <code>free</code>	предупреждает	Aborted (core dumped)
выход за границу блока	молчит	повреждение чужой памяти

Две из четырёх ошибок проходят совершенно бесшумно и завершаются нормально. Программа с любой из них может годами выдавать правильные ответы и упасть при первом изменении в соседнем файле.

Главное

Собирайте с санитайзером всегда, кроме замеров скорости. `make` на этом курсе делает это за вас.

РАЗДЕЛ 3

Рост

realloc

```
int *bigger = realloc(a, new_cap * sizeof *bigger);  
if (bigger == NULL) { /* a ещё жив, блок не тронут */ }  
a = bigger;
```

`realloc` меняет размер уже выделенного блока. Он либо расширяет его на месте, либо выделяет новый, **переносит содержимое сам** и освобождает старый. Что именно произошло, видно только по возвращённому адресу.

Два частных случая избавляют от отдельной ветки для первого роста: `realloc(NULL, n)` равносильно `malloc(n)`, а `free(NULL)` ничего не делает.

Почему не `a = realloc(a, ...)`

При неудаче `realloc` возвращает `NULL`, оставляя старый блок нетронутым. Запись прямо в `a` затрёт единственный адрес этого блока: произойдёт утечка памяти.

Блок переезжает

```
cap 2 -> 4, было по адресу 0x558256f76010
           стало по адресу 0x558256f76010
cap 4 -> 8, было по адресу 0x558256f76010
           стало по адресу 0x558256f77150
```

Первый рост обошёлся без переезда, второй — нет. Предсказать это нельзя, и рассчитывать на сохранение адреса нельзя тоже.

Все прежние указатели мертвы

После переезда любой указатель внутри старого блока — висячий: и сохранённый раньше `int *first = a`, и указатель на отдельный элемент. Живёт только то, что вернул `realloc`.

Собранная с санитайзером программа переезжает при каждом `realloc`: так ошибка обнаруживается на первом же запуске.

Сколько просить

Место кончилось на `cap` элементах, нужен ещё один. Сколько выделять?

стратегия	новых блоков	перенесено суммарно	на элемент
<code>cap + 1</code>	20000	199 990 000	9999.5
<code>cap + 8</code>	2500	24 990 000	1249.5
<code>cap * 2</code>	14	32 764	1.6
как в CPython	53	170 688	8.5

Двадцать тысяч элементов, добавленных по одному; перенесённые элементы посчитаны программой `10-growth.c`.

При ста тысячах элементов первая строка даёт 49999.5 на элемент, а третья — 1.3. Два числа в последнем столбце растут вместе с `n`, два других не растут.

Откуда берётся константа

Вместимость начинается с c и умножается на r . Переезд переносит ровно ту вместимость, которая кончилась, поэтому всего переносится

$$c + cr + cr^2 + \dots + \frac{C}{r} = \frac{C - c}{r - 1},$$

где c — вместимость, на которой всё закончилось. Сумма выражается через последний член: работа пропорциональна c , а не числу слагаемых.

c не меньше n и меньше rn , поэтому на добавленный элемент приходится от $1/(r - 1)$ до $r/(r - 1)$ переносов.

	от	до	измерено
$\text{cap} * 2, r = 2$	1	2	1.31 и 1.64
CPython, $r \approx 1.125$	около 8		7.98 и 8.53

Начальная вместимость c считается малой рядом с n ; формула CPython геометрическая лишь примерно.

Что значит «амортизированная»

Определение

Операция стоит амортизированную $O(1)$, если любые n подряд идущих операций вместе стоят $O(n)$. Про отдельную операцию при этом не утверждается ничего.

Отдельный `append` может оказаться очень дорогим: тот, что попал на переезд миллионного массива, переносит миллион элементов. Дорогих вызовов мало, и чем они дороже, тем реже случаются, — поэтому среднее по любой последовательности остаётся ограниченным.

Слово «амортизированная» не означает «в среднем по случайным данным». Здесь нет никакой случайности и никакого предположения о входе: оценка верна для любой последовательности вызовов.

РАЗДЕЛ 4

Свой вектор

Три числа, которые описывают вектор

Указатель, длина и вместимость описывают один объект, и передавать их порознь неудобно. `struct` складывает их рядом:

```
struct Vec {  
    int    *data;    // блок в куче или NULL  
    size_t  len;     // сколько элементов занято  
    size_t  cap;     // сколько помещается без переезда  
};  
  
struct Vec v = {NULL, 0, 0};    // пустой вектор  
printf("%zu %zu\n", v.len, sizeof v);    // 0 24
```

Поля читаются через точку: `v.len`. Если в руках указатель на структуру, пишется стрелка: `p->len` — сокращение для `(*p).len`.

Как `struct` устроена в памяти и почему её размер равен 24 — завтра. Сегодня она нужна как способ держать три переменные вместе.

Добавление в конец

```
bool vec_push(struct Vec *v, int value)
{
    if (v->len == v->cap) {
        size_t new_cap = (v->cap == 0) ? 4 : v->cap * 2;
        int *bigger = realloc(v->data, new_cap * sizeof *bigger);
        if (bigger == NULL)
            return false;           // вектор остался прежним и корректным
        v->data = bigger;
        v->cap = new_cap;
    }
    v->data[v->len++] = value;
    return true;
}
```

Вся амортизация — в строке с `new_cap`; остальное выполняется за фиксированное время. Четвёрка нужна потому, что ноль при умножении на два остался бы нулём.

Остальные операции

```
int  vec_get(const struct Vec *v, size_t i) { return v->data[i]; }
void vec_free(struct Vec *v) {
    free(v->data);
    v->data = NULL;  v->len = 0;  v->cap = 0;
}
```

`vec_free` обнуляет поля не из аккуратности: после него структура описывает корректный пустой вектор, и повторный вызов ничего не ломает.

Вставка и удаление в середине — вчерашний сдвиг участка, применённый к блоку в куче:

```
/* удалить элемент pos: сдвинуть хвост влево на одну позицию */
move_range(v->data + pos, v->data + pos + 1, v->len - pos - 1);
v->len--;
```

`move_range` — та самая функция, которую вы написали вчера в задаче 4.

Во что обходятся операции

	ваш вектор	список в Python
чтение по индексу	$O(1)$	<code>a[i]</code> — $O(1)$
запись по индексу	$O(1)$	<code>a[i] = x</code> — $O(1)$
добавление в конец	амортизированная $O(1)$	<code>append</code> — та же
удаление с конца	$O(1)$	<code>pop()</code> — та же
вставка в начало	$O(n)$	<code>insert(0, x)</code> — та же
удаление из начала	$O(n)$	<code>pop(0)</code> — та же
поиск значения	$O(n)$	<code>x in a</code> — та же

Столбцы совпадают целиком, потому что внутри CPython лежит ровно эта структура. Разница между `append` и `insert(0, x)` — разница между записью в готовую ячейку и сдвигом всего массива.

Через два дня к таблице добавится строка про `dict`, и она будет устроена иначе.

РАЗДЕЛ 5

Ответ на вопрос дня

Как это сделано в CPython

```
/* listobject.c, функция list_resize */  
new_allocated = ((size_t)newsize + (newsize >> 3) + 6) & ~(size_t)3;
```

Новая вместимость — это нужная длина плюс её восьмая часть плюс шесть, округлённые вниз до кратного четырём. Множитель получается около 1.125.

Формула проверяется прямо из интерпретатора: `sys.getsizeof` даёт 56 байт на сам объект плюс 8 байт на каждую ячейку.

длина, при которой растёт	1	5	9	17	25	33	41	53
<code>getsizeof</code>	88	120	184	248	312	376	472	568
вместимость	4	8	16	24	32	40	52	64

Третья строка — это $(\text{getsizeof} - 56) / 8$, и она совпадает с формулой на всех восьми значениях.

Почему множитель именно такой

Восьмикратная работа выбрана сознательно: взамен список никогда не занимает вдвое больше памяти, чем в нём лежит.

	переносов на элемент	лишняя память
<code>cap * 2</code>	от 1 до 2	до 100%
CPython, ≈ 1.125	около 8	до 12%

Копирование подряд идущей памяти обходится дёшево, а лишние мегабайты в программе, где списков тысячи, — дорого. Обе стратегии дают амортизированную $O(1)$; Python выбрал ту, у которой меньше расход памяти.

Та же функция `list_resize` умеет и сжимать список: после удалений, оставивших занятой меньше половины вместимости, она выделяет блок поменьше.

Почему append — амортизированная $O(1)$

Список — это блок в куче, длина и вместимость. Пока вместимость не исчерпана, `append` записывает указатель в готовую ячейку за фиксированное время. Когда исчерпана, выделяется блок, больший в постоянное число раз, и всё содержимое переносится. Такие переезды случаются тем реже, чем они дороже, и сумма перенесённого по n добавлениям остаётся пропорциональной n .

Отсюда следуют вещи, которые обычно запоминают отдельно:

- `insert(0, x)` и `pop(0)` стоят $O(n)$ — они двигают весь массив;
- `a[i]` стоит $O(1)$ при любом `i` — это адресная арифметика;
- `append` в цикле на миллион элементов делает около миллиона записей и порядка сотни переездов.

Итог дня

- `malloc` даёт блок нужного размера во время работы, `free` возвращает его. Размер блока не хранится нигде, кроме вашей программы.
- У блока один владелец. Утечка, обращение после `free`, двойной `free` и выход за границу ловятся санитайзером; две из этих четырёх ошибок без него не видны совсем.
- `realloc` может перенести блок. После него живёт только возвращённый адрес, а результат кладётся во временную переменную.
- Рост в постоянное число раз даёт амортизированную $O(1)$: сумма переносов пропорциональна длине, а не её квадрату.
- Питоновский список — блок, длина и вместимость. Вы напишете такой же.

Задачи дня: `day3-heap-vector/tasks/`

Вопросы к защите открыты: `defense/questions.md`, посмотрите их до того, как начнёте решать.